TALLER PRÁCTICO 2

EcoMarket

Plataforma web para comercio sostenible construida con inteligencia artificial

HTML • CSS • JavaScript • Prompt Engineering • JSON • DOM • localStorage

Estudiante: Andrés Garnica
Asignatura: Desarrollo web con IA
Repositorio: github.com/andhresgc-hash/Taller_Ecomarket

Contenido

1. Resumen del proyecto
2. Fase 1 — Landing page de EcoMarket
3. Fase 2 — Dashboard administrativo
4. Fase 3 — Diseño y generación del dataset
5. Fase 4 — Validación programática
6. Fase 5 — Visualización, CRUD y localStorage
7. Prompts utilizados, técnicas y ajustes
8. Entregables del taller
9. Preguntas de reflexión
10. Conclusiones

1. Resumen del proyecto

EcoMarket es una startup colombiana que conecta a pequeños productores locales con personas que quieren comprar de forma responsable. En este taller construí un prototipo con cinco partes: una landing page, un dashboard administrativo, un dataset de 35 productos generado con IA, una función que valida esos datos y una aplicación para administrar el catálogo con búsqueda, filtros, CRUD, localStorage y un gráfico.

Usé solo HTML5, CSS3 y JavaScript vanilla, sin frameworks. La única librería externa es Chart.js, que el taller permite para el gráfico.

1.1 Repositorio en GitHub

Código: github.com/andhresgc-hash/Taller_Ecomarket

Cada nombre de la tabla es un enlace al archivo en GitHub.

Archivo	Responsabilidad
index.html	Landing page de EcoMarket (HTML semántico en español).
dashboard.html	Aplicación de administración: sidebar, KPI, filtros, tarjetas, tabla, gráfico y modal.
styles.css	Estilos de la landing y del dashboard con variables en :root, Grid/Flexbox y media queries.
app.js	Menú hamburguesa, carga de datos, renderizado, filtros, CRUD, KPI, gráfico y localStorage.
validacion.js	Función validarProductos(lista) con las reglas de negocio del dataset.
productos.json	Dataset inicial de 35 productos (5 manuales y 30 generados con IA).
productos-ejemplo.js	Los cinco registros manuales usados como ejemplos few-shot.
evidencia/	Dataset antes y después de corregir y dataset con errores de prueba.
capturas/	Capturas de pantalla del proyecto.
documentacion/	Este documento en formato Word.

2. Fase 1 — Landing page de EcoMarket

2.1 Priming inicial

Abrí una conversación nueva y le di a la IA el rol de desarrollador frontend senior de comercio electrónico, con el contexto de EcoMarket, y le dije que todavía no escribiera código. La IA confirmó su rol y me hizo preguntas sobre las secciones, la identidad visual y las herramientas. Yo las respondí antes de pasar al diseño.

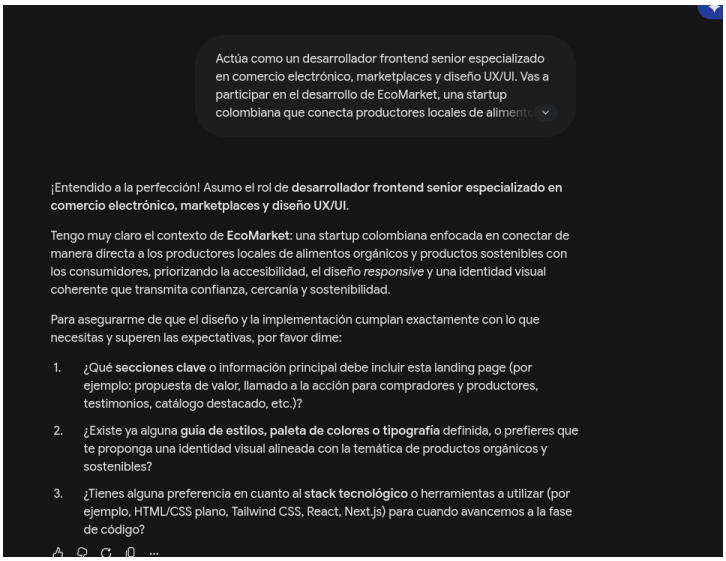


Figura 1. Respuesta de la IA al prompt de priming: confirma el rol y pregunta por la información que le falta.

> **Técnica aplicada:** priming (darle un rol y un contexto a la IA) y meta-prompting para que ella misma dijera qué información le faltaba.

2.2 Propuesta de diseño con Chain of Thought

Le pedí a la IA una propuesta de diseño en seis puntos, sin código. Usé Chain of Thought para que explicara su razonamiento paso a paso: qué busca el usuario, cómo llevarlo a comprar, accesibilidad y coherencia de marca. Esta fue la propuesta que revisé y aprobé.

Secciones y orden de lectura: header, hero con dos botones, beneficios de comprar local, categorías, productores aliados, productos destacados, testimonios, suscripción y footer. El hero se lee en forma de Z y el resto de la página en forma de F.

Paleta (regla 60/30/10):

Rol	Color	Hex	Justificación
Dominante (60 %)	Marfil orgánico	#F9F8F3	Fondo cálido, evita el deslumbramiento del blanco puro.
Secundario (30 %)	Verde bosque	#1E3F20	Naturaleza y solidez; contraste superior a 7:1 sobre fondo claro.
Texto	Gris pizarra	#2D3748	Texto de cuerpo; contraste AAA sobre el marfil (11,5:1).
Acento CTA (10 %)	Terracota	#D96B43	Evoca la tierra colombiana; contraste AA con texto blanco (4,6:1).
Acento suave	Verde menta	#E2F0D9	Fondos de tarjetas e insignias de orgánico.

Tipografía: elegí Outfit para los títulos e Inter para el texto, con base de 16 px (H1 de 44 px en computador y 32 px en celular; H2 de 32 y 24 px).

Accesibilidad y responsive: cuidé que el contraste fuera mínimo 4,5:1, que el foco se viera con un borde de 2 px, que las imágenes tuvieran texto alternativo y cada campo su etiqueta, y que se pudiera navegar con teclado. El diseño es mobile-first con media queries en 640 y 1024 px.

2.3 Generación de HTML y CSS

Con la propuesta aprobada pedí los archivos por separado (zero-shot) y escribí los requisitos técnicos: HTML5 semántico, lang="es", IDs descriptivos, clases en kebab-case, nada de CSS ni JavaScript dentro del HTML, variables en :root, Flexbox o Grid, mobile-first y estados hover y focus.



Figura 2. Landing page de EcoMarket (sección principal), ejecutada desde el repositorio.

3. Fase 2 — Dashboard administrativo

3.1 Role prompting y diseño

Le di a la IA el rol de diseñador UX/UI de dashboards y le pedí que describiera la distribución, los colores, la tipografía y los íconos antes de hacer código. Quedó un sidebar fijo de 260 px en verde bosque y un área principal con barra superior, bienvenida, botón de acción, tarjetas KPI y la tabla de inventario. Mantuve la paleta de la landing y usé verde para Disponible, amarillo para Stock bajo y rojo para Agotado.

3.2 Tarjetas KPI y tabla de productos

Pedí tres KPI (productos registrados, inventario bajo y valor del inventario en COP) y una tabla de siete columnas con cinco filas de ejemplo (ECO001 a ECO005), con badges de estado, filas alternadas y botones de editar y eliminar.

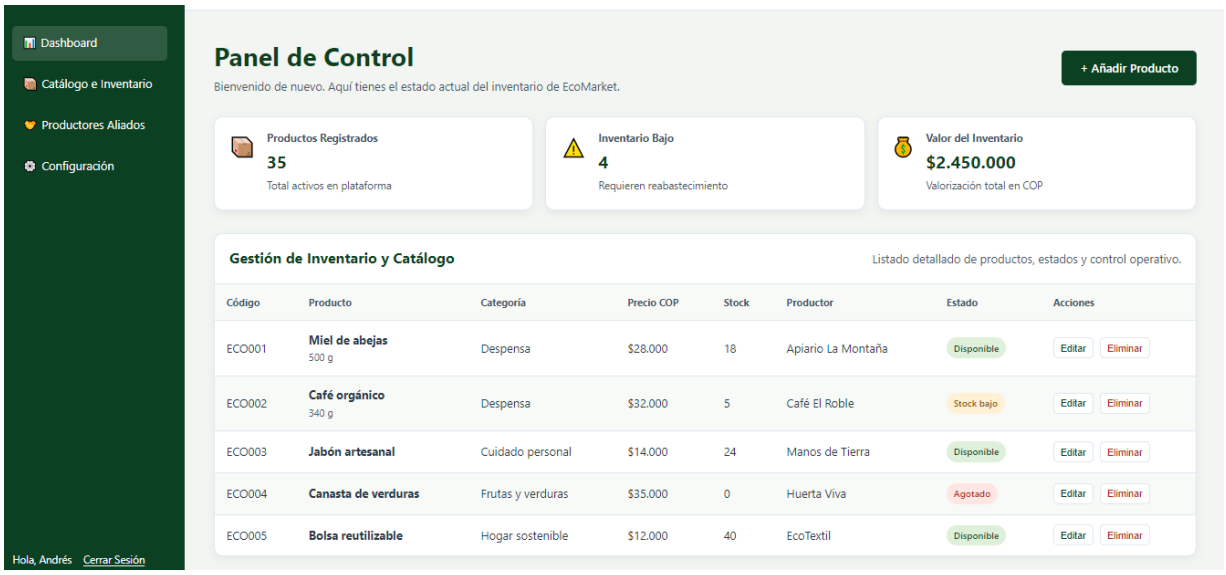


Figura 3. Dashboard con sidebar, tres KPI y tabla de cinco productos con badges de estado.

3.3 Menú hamburguesa con JavaScript

El menú lo hice con JavaScript vanilla y una transición CSS de 0,3 s. El botón cambia entre ☰ y ×, actualiza aria-expanded, y el menú se cierra al hacer clic afuera, al elegir una opción, al presionar Escape o al agrandar la pantalla.

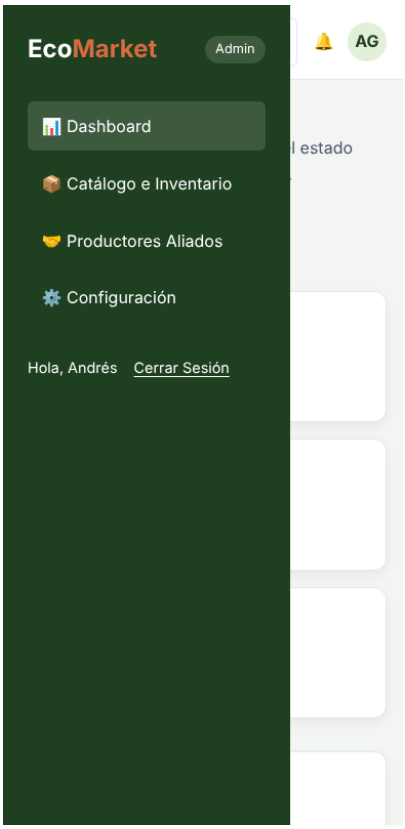


Figura 4. Vista móvil (390 px) con el menú abierto y el botón convertido en ×.

> **Ajuste que hice:** el KPI «Inventario bajo» cuenta los productos con stock menor o igual al stock mínimo, como dice el taller. Por eso también cuenta los agotados, porque un producto con stock 0 también está por debajo de su mínimo.

4. Fase 3 — Diseño y generación del dataset

4.1 Estructura de datos

Campo	Tipo	Restricciones	Ejemplo
id	Entero	Único, mayor que 0	1
nombre	Texto	3 a 80 caracteres	Miel de abejas 500 g
categoria	Categórico	Una de las 4 categorías	Despensa
precio	Numérico	Entero mayor que 0 (COP)	28000
stock	Entero	Mayor o igual a 0	18
stockMinimo	Entero	Mayor o igual a 0	5
productor	Texto	3 a 80 caracteres	Apiario La Montaña
ciudad	Categórico	8 ciudades válidas	Bucaramanga

Campo	Tipo	Restricciones	Ejemplo
organico	Booleano	true / false	true
fechaRegistro	Fecha	YYYY-MM-DD entre 2026-01-01 y 2026-11-03	2026-10-15
activo	Booleano	true / false	true

Categorías válidas: Frutas y verduras, Despensa, Cuidado personal y Hogar sostenible.
Ciudades válidas: Bucaramanga, Floridablanca, Girón, Bogotá, Medellín, Cali, Tunja y Cúcuta.

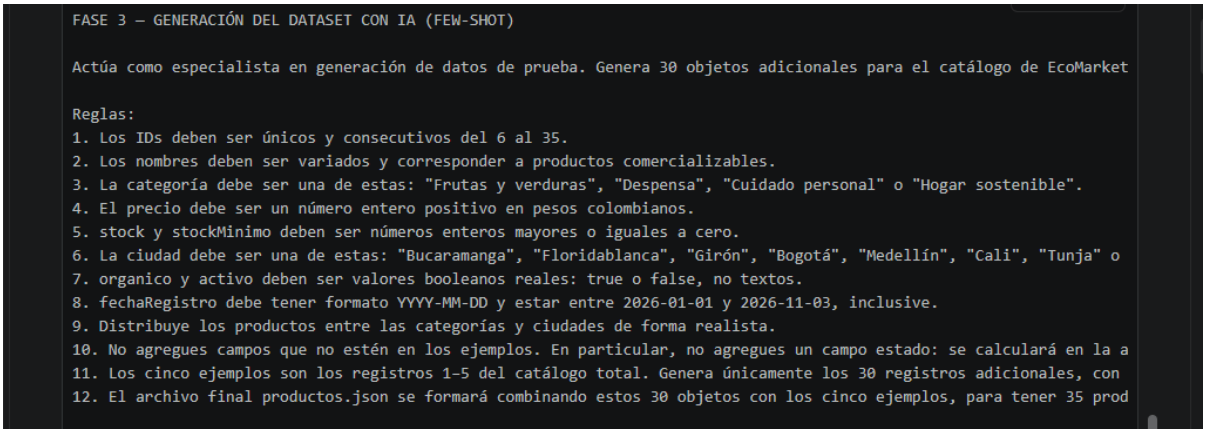
4.2 Cinco registros manuales

Antes de usar la IA escribí a mano cinco productos en productos-ejemplo.js. Me sirvieron para entender la estructura y como ejemplos para el prompt few-shot.

ID	Nombre	Categoría	Precio	Stock	Productor · Ciudad
1	Miel de abejas 500 g	Despensa	$28.000	18	Apiario La Montaña · Bucaramanga
2	Café orgánico 340 g	Despensa	$32.000	5	Café El Roble · Bogotá
3	Jabón artesanal de caléndula	Cuidado personal	$14.000	24	Manos de Tierra · Medellín
4	Canasta de verduras orgánicas	Frutas y verduras	$35.000	0	Huerta Viva · Floridablanca
5	Bolsa reutilizable de algodón	Hogar sostenible	$12.000	40	EcoTextil · Cali

4.3 Generación con few-shot prompting

Le pedí a la IA que actuara como especialista en datos de prueba y generara 30 productos más, con IDs del 6 al 35, siguiendo la estructura de mis cinco ejemplos y doce reglas. Le dije que devolviera solo un arreglo JSON válido y que no inventara un campo «estado», porque ese se calcula en la aplicación.



```
FASE 3 — GENERACIÓN DEL DATASET CON IA (FEW-SHOT)

Actúa como especialista en generación de datos de prueba. Genera 30 objetos adicionales para el catálogo de EcoMarket

Reglas:
1. Los IDs deben ser únicos y consecutivos del 6 al 35.
2. Los nombres deben ser variados y corresponder a productos comercializables.
3. La categoría debe ser una de estas: "Frutas y verduras", "Despensa", "Cuidado personal" o "Hogar sostenible".
4. El precio debe ser un número entero positivo en pesos colombianos.
5. stock y stockMinimo deben ser números enteros mayores o iguales a cero.
6. La ciudad debe ser una de estas: "Bucaramanga", "Floridablanca", "Girón", "Bogotá", "Medellín", "Cali", "Tunja" o
7. organico y activo deben ser valores booleanos reales: true o false, no textos.
8. fechaRegistro debe tener formato YYYY-MM-DD y estar entre 2026-01-01 y 2026-11-03, inclusive.
9. Distribuye los productos entre las categorías y ciudades de forma realista.
10. No agregues campos que no estén en los ejemplos. En particular, no agregues un campo estado: se calculará en la a
11. Los cinco ejemplos son los registros 1–5 del catálogo total. Genera únicamente los 30 registros adicionales, con
12. El archivo final productos.json se formará combinando estos 30 objetos con los cinco ejemplos, para tener 35 prod
```

Figura 5. Prompt few-shot con las reglas de generación del dataset.

El resultado, productos.json, quedó con 35 registros distribuidos así:

Categoría	Productos	Ciudad	Productos
Despensa	11	Bogotá / Medellín / Tunja	6 c/u
Frutas y verduras	9	Bucaramanga / Cali	5 c/u
Cuidado personal	8	Floridablanca / Cúcuta	3 c/u
Hogar sostenible	7	Girón	1

5. Fase 4 — Validación programática

5.1 La función validarProductos(lista)

validacion.js recibe un arreglo y devuelve una lista de errores. Cada error dice el ID del producto, el campo y qué está mal. Revisa lo siguiente:

- IDs únicos, enteros y mayores que 0.
- Nombre y productor de 3 a 80 caracteres.
- Categoría y ciudad dentro de las listas permitidas.
- Precio entero positivo; stock y stockMinimo enteros mayores o iguales a 0.
- organico y activo con tipo booleano real.
- fechaRegistro con formato YYYY-MM-DD, fecha real y dentro del rango 2026-01-01 a 2026-11-03.

La aplicación usa esta función en tres momentos: al cargar productos.json, al leer lo guardado en localStorage y antes de guardar cualquier cambio del CRUD. Si hay errores, los muestra en pantalla y no guarda nada.

5.2 Prueba de la validación con errores controlados

Para comprobar que cada regla funciona, copié el dataset y le metí diez errores a propósito, uno por cada tipo de regla. Al ejecutar validarProductos() detectó los diez.

ID	Campo	Error introducido	Mensaje de la validación
7	categoria	«Lácteos» (no existe)	La categoría «Lácteos» no es válida.
9	id	ID duplicado	El ID 9 está repetido.
12	precio	«28000» como texto	El precio debe ser un número entero mayor que 0.
14	stock	-4	El stock debe ser un entero mayor o igual a 0.
16	ciudad	«Pereira»	La ciudad no pertenece a las permitidas.
18	organico	«true» como texto	El campo organico debe ser booleano.
20	fechaRegistro	2026-12-15	La fecha debe estar entre 2026-01-01 y 2026-11-03.
22	nombre	«Ab» (2 caracteres)	El nombre debe tener entre 3 y 80 caracteres.

ID	Campo	Error introducido	Mensaje de la validación
24	stockMinimo	2,5	El stock mínimo debe ser un entero mayor o igual a 0.
26	activo	1 en lugar de true	El campo activo debe ser booleano.

5.3 Meta-validación con IA

Usé el prompt del taller: la IA actúa como auditor de calidad de datos, revisa el JSON de los 35 productos y presenta un informe en tabla sin modificar nada. Se lo pedí a Claude con mi productos.json y comparé el informe con el resultado de validarProductos(), que daba 0 errores. Es decir, los datos cumplían las reglas del código, pero la IA encontró problemas de calidad que la función no puede ver.

5.4 Documentación de errores

Error encontrado	Registro(s) afectado(s)	Causa probable	Corrección aplicada
Unidades sin espacio («1kg», «500g», «250ml», «1L»)	14 productos (IDs 6, 7, 10, 11, 12, 16, 19, 21, 22, 25, 30, 31, 32, 35)	La IA no siguió el formato del ejemplo «500 g»	Se agregó el espacio: «1 kg», «500 g», «250 ml», «1 L»
Todos los productos con organico: true, incluso esponja, brocha, bloqueador y papel higiénico	35 de 35; objetos no alimenticios: IDs 20, 29, 33, 34	La IA copió el valor del ejemplo	organico pasó a false en los IDs 20, 29, 33 y 34
Concordancia de género: «Zanahoria orgánico»	ID 22	Error de redacción de la IA	«Zanahoria orgánica 1 kg»
Tilde faltante: «Trapiche San Jose»	ID 11	Error de escritura de la IA	«Trapiche San José»
Productores repetidos (AgroSuroeste en 3 productos; otros 5 en 2)	IDs 1 y 24, 4 y 22, 6, 25 y 31, 8 y 18, 9 y 29, 19 y 35	Poca variedad en la generación	No se corrige: un productor puede vender varios productos. Se deja como observación.
Un mismo productor en dos ciudades: «Frutas Exóticas» (Cali y Tunja)	IDs 19 y 35	Distribución aleatoria de ciudades	No se corrige: puede tener dos sedes. Se deja como observación.

Evidencia antes y después. Guardé las dos versiones del dataset en la carpeta evidencia/ del repositorio: productos_antes_de_corregir.json y el dataset corregido productos.json. Ejecuté validarProductos() en las dos y dieron 0 errores. La prueba con errores a propósito de la sección 5.2 está en productos_con_errores.json.

5.5 Resultado sobre el dataset final

> **Resultado:** validarProductos() sobre productos.json devuelve 0 errores en los 35 registros.

Con esto vi que no basta con que la IA revise los datos. Una función en JavaScript siempre detecta los errores de tipo, de rango y de listas, y eso a simple vista se me puede pasar.

6. Fase 5 — Visualización, CRUD y localStorage

6.1 Renderizado dinámico

Los productos los muestro con JavaScript manipulando el DOM (createElement y replaceChildren). Hay dos vistas que se pueden cambiar: tarjetas en un grid CSS y tabla. Cada producto muestra nombre, categoría, ciudad, precio en COP, stock, productor, un badge de estado y los botones Editar y Eliminar.

El estado se calcula así: Agotado si stock = 0; Stock bajo si stock > 0 y stock ≤ stockMinimo; Disponible si stock > stockMinimo.

6.1.1 Funciones clave de app.js

Función	Qué hace en el proyecto
cargarProductos()	Lee localStorage; si no hay datos guardados, carga productos.json con fetch() y valida el catálogo.
guardarProductos(lista)	Valida y serializa el arreglo con JSON.stringify y lo guarda en localStorage; luego vuelve a renderizar.
renderizarProductos(lista)	Construye las tarjetas y las filas de la tabla en el DOM y actualiza el contador y los KPI.
filtrarProductos()	Aplica búsqueda y filtros combinados sobre el arreglo original de productos.
abrirModal(id)	Abre el formulario; si recibe un producto o su ID, precarga los datos para editar.
guardarProducto()	Valida el formulario, agrega o actualiza el producto y persiste los cambios.
eliminarProducto(id)	Elimina el producto, guarda los cambios y vuelve a renderizar (con confirmación previa).
actualizarKPIs()	Recalcula productos registrados, inventario bajo y valor del inventario.
actualizarGrafico()	Recalcula el conteo por categoría y actualiza el gráfico de Chart.js.

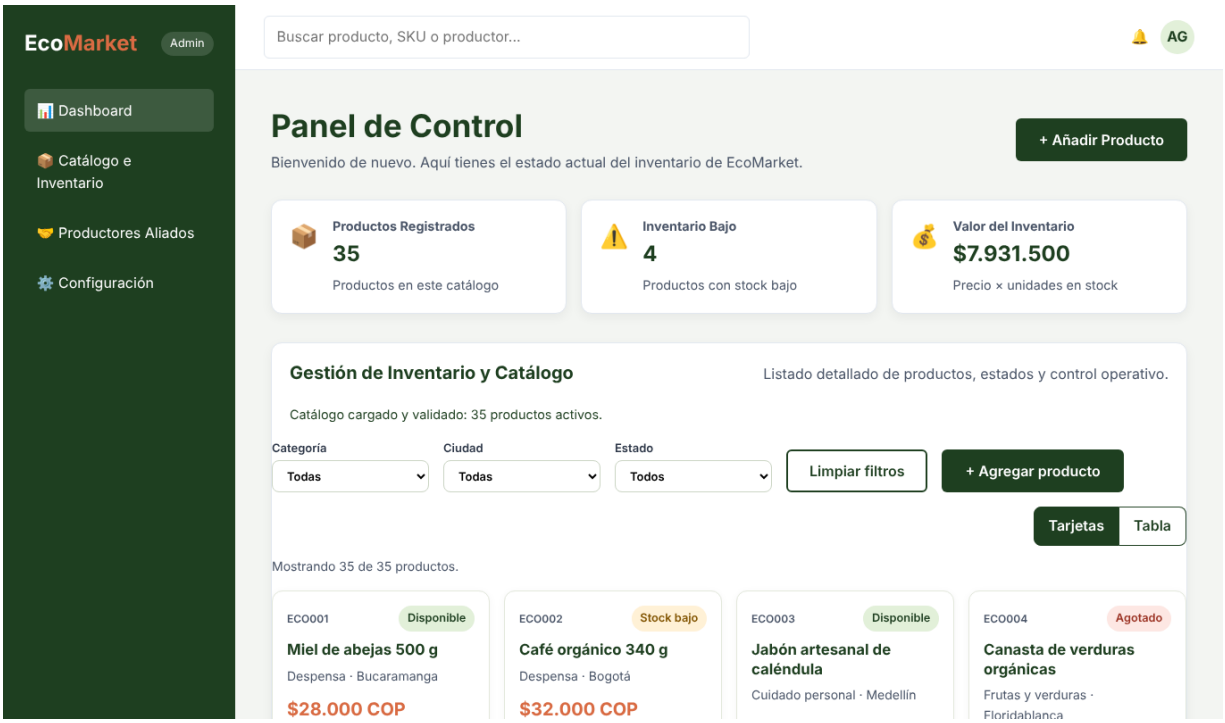


Figura 6. Dashboard con KPI calculados, filtros y vista de tarjetas con 35 productos.

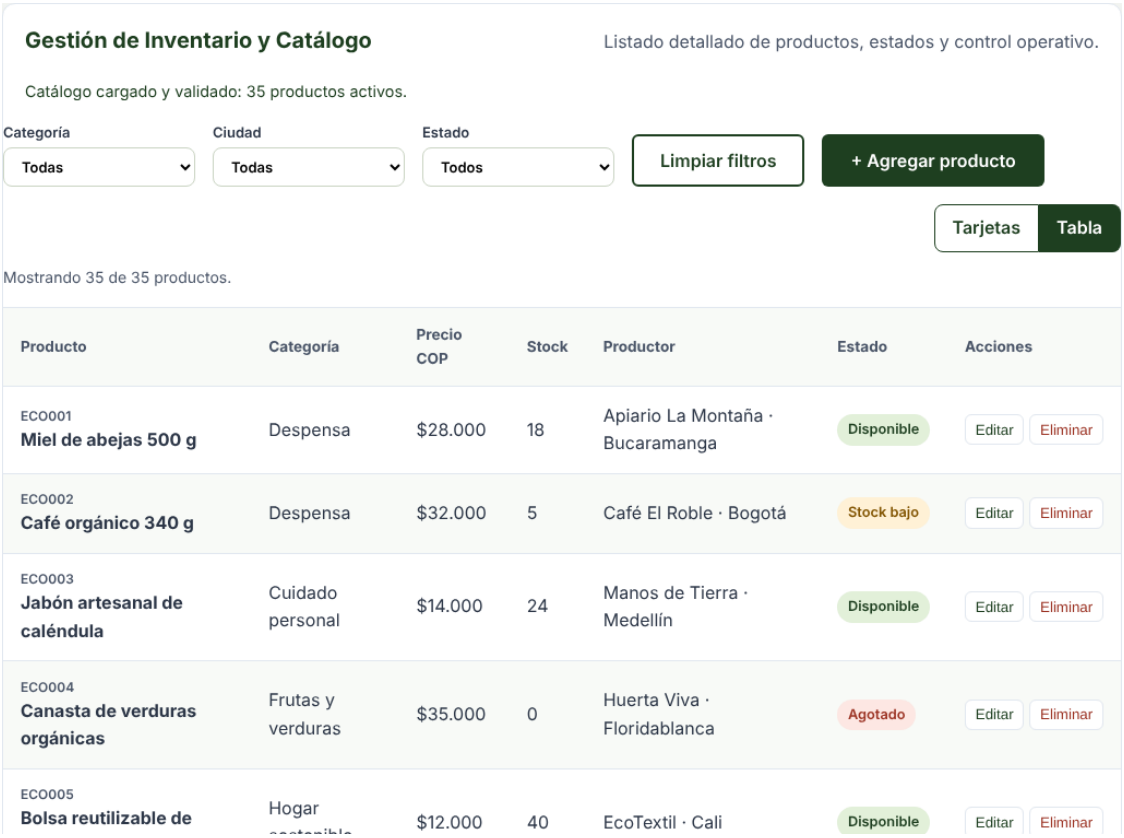


Figura 7. Vista de tabla de inventario con badges, filas alternadas y acciones.

6.2 Búsqueda y filtros combinados

La búsqueda en tiempo real, los selectores de categoría, ciudad y estado y el botón «Limpiar filtros» trabajan siempre sobre la lista completa de productos con la función filtrarProductos(). Así los filtros se pueden combinar y no se pierden productos al cambiar uno. Debajo aparece «Mostrando X de Y productos» y un mensaje cuando no hay resultados.

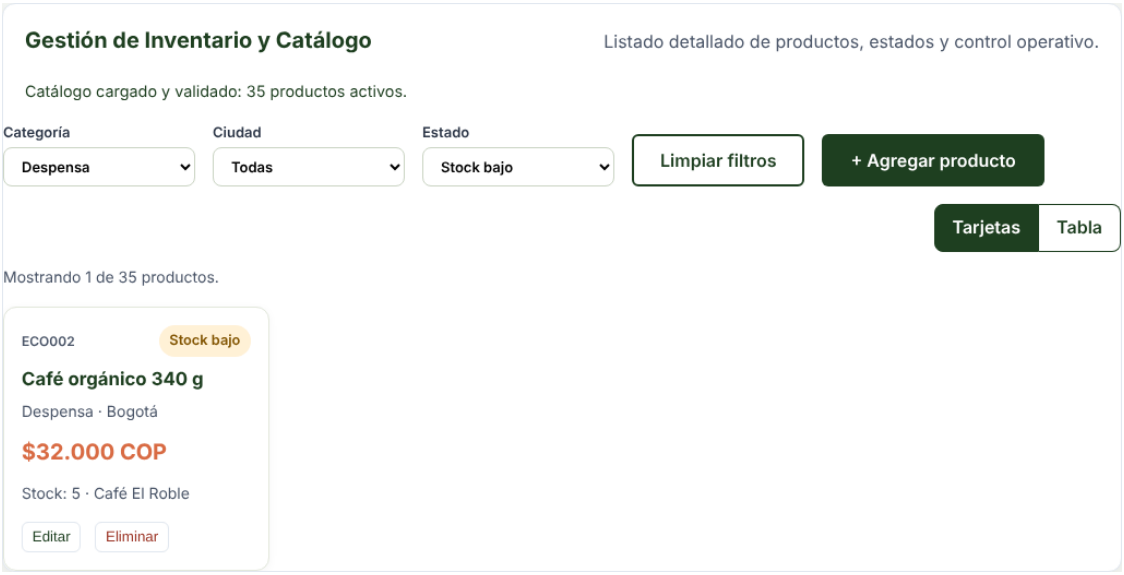


Figura 8. Filtros combinados: categoría Despensa + estado Stock bajo devuelve 1 de 35 productos.

6.3 Operaciones CRUD

- **Crear:** los botones «Añadir producto» y «Agregar producto» abren un modal con formulario. Trae valores por defecto (ciudad, stock mínimo y fecha), tiene el botón «Guardar y agregar otro», muestra un aviso al guardar y resalta el producto nuevo en la lista.
- **Leer:** los productos se muestran en tarjetas y en tabla.
- **Actualizar:** Editar abre el modal con los datos del producto.
- **Eliminar:** pide confirmación antes de borrar.

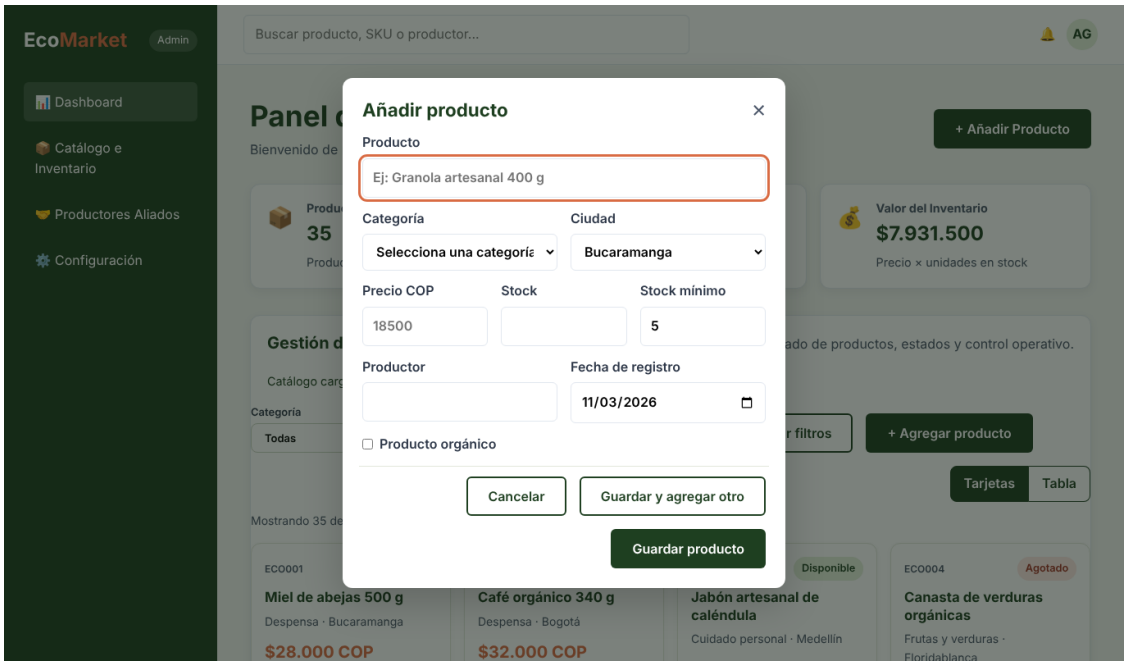


Figura 9. Modal «Añadir producto» con campos obligatorios, rangos y listas de categorías y ciudades.

6.4 Persistencia con localStorage

Al iniciar, la aplicación carga productos.json y, si hay cambios guardados en localStorage, usa esos. Después de cada operación del CRUD se valida la lista, se guarda y se vuelve a mostrar todo sin recargar la página.

Probé el flujo completo con un navegador automatizado:

Acción	Resultado observado
Carga inicial	35 productos y KPI actualizados
Filtro categoría Despensa + estado Stock bajo	1 de 35 productos; «Limpiar filtros» vuelve a 35
Agregar «Granola artesanal 400 g»	36 productos; KPI de productos = 36
Recargar la página	Siguen los 36 productos (persistencia correcta)
Eliminar el producto agregado	35 productos; KPI y valor del inventario se recalculan
Errores de JavaScript en consola	Ninguno

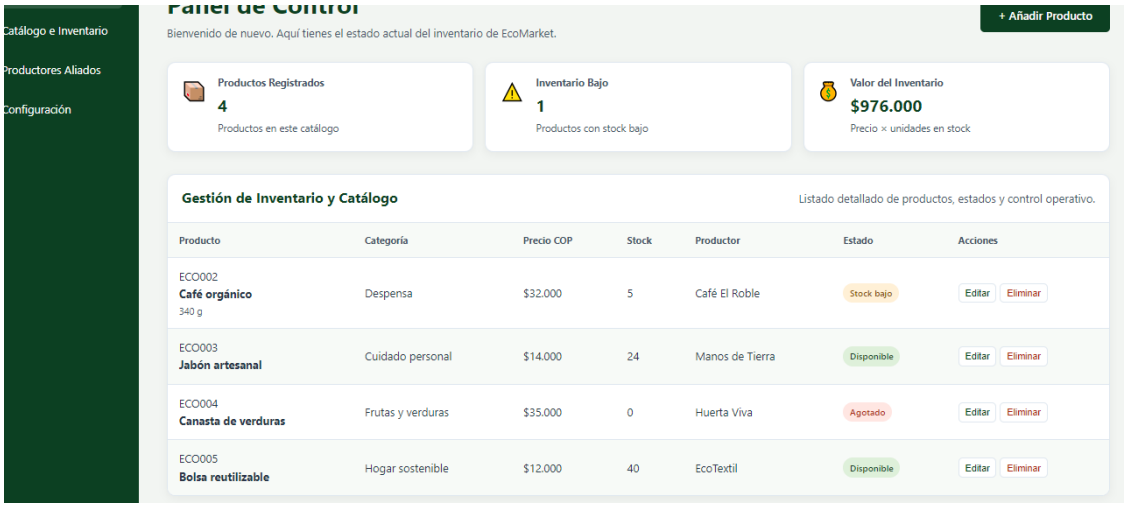


Figura 10. KPI recalculados después de eliminar productos (4 productos, valor $976.000).

6.5 Gráfico estadístico (bonus)

Con Chart.js hice un gráfico de barras con la cantidad de productos por categoría. La función actualizarGrafico() vuelve a contar y actualiza el gráfico cada vez que se agrega, edita o elimina un producto. Con los datos iniciales muestra 9 productos en Frutas y verduras, 11 en Despensa, 8 en Cuidado personal y 7 en Hogar sostenible.



Figura 11. Gráfico de barras «Productos por categoría» en el dashboard (Chart.js).

7. Prompts utilizados, técnicas y ajustes

Fase	Técnica	Prompt (resumen)	Ajustes realizados
1	Priming + meta-prompting	Rol de desarrollador frontend senior de e-commerce; confirmar contexto y preguntar lo que falte, sin código.	Respondí lo que la IA preguntó (secciones, identidad visual, herramientas).
1	Chain of Thought	Analizar y proponer diseño en seis puntos, sin código.	Revisé el contraste, las llamadas a la acción y las secciones antes de pedir el código.
1	Zero-shot	index.html y styles.css por separado con requisitos técnicos.	Revisé la semántica, el diseño responsive y la accesibilidad.
2	Role prompting	Diseñador UX/UI de dashboards; describir layout, colores y tipografía.	Adapté la propuesta a tres KPI y siete columnas de tabla.
2	Zero-shot con requisitos	Tarjetas KPI, tabla de cinco productos y menú hamburguesa en JS vanilla.	Cambié el KPI «Inventario bajo» para que cuente stock ≤ stockMinimo.
3	Few-shot	30 objetos adicionales con 12 reglas y cinco ejemplos manuales.	Verifiqué que no tuviera campo «estado» y que mis cinco ejemplos fueran los IDs 1 a 5.
4	Meta-validación	Auditor de calidad de datos que reporta en tabla sin modificar el dataset.	Corregí 18 registros (unidades, organico, tilde y concordancia) y además usé validarProductos(), porque no quise depender solo de la IA.
5	Prompts por fase	Renderizado, filtros, CRUD, localStorage y gráfico en prompts separados.	Agregué filtros combinados, vista de tarjetas y gráfico, y probé todo el flujo.

8. Entregables del taller

Entregable	Dónde está
Landing page funcional: index.html y styles.css	index.html en el repositorio
Dashboard administrativo con la identidad visual	dashboard.html en el repositorio
productos.json con 5 registros manuales y 30 con IA	productos.json en el repositorio
validacion.js y evidencia del reporte	validacion.js y carpeta evidencia/
Aplicación CRUD: app.js, estilos y HTML	app.js en el repositorio
Capturas de pantalla	carpeta capturas/
Documento con prompts, técnicas y ajustes	Sección 7 de este documento
Repositorio en GitHub	github.com/andhresgc-hash/Taller_Ecomarket

9. Preguntas de reflexión

1. ¿Qué diferencias encontraste entre zero-shot, one-shot y few-shot prompting?

En zero-shot solo le di la instrucción a la IA, y funcionó bien para el HTML y el CSS. En one-shot y few-shot le muestro uno o varios ejemplos para que copie el formato. Para el dataset usé few-shot con mis 5 productos manuales y los 30 que generó quedaron con los mismos campos y tipos. Con ejemplos el resultado es más exacto, aunque el prompt queda más largo.

2. ¿Por qué es importante definir la estructura y las reglas del dataset antes de generarlo con IA?

Porque si no, la IA inventa cosas: campos que no existen, ciudades o categorías que no están en la lista, o números escritos como texto. Al definir antes los campos, los tipos y las listas válidas, le pedí justo lo que necesitaba y después pude comprobar que los datos estuvieran bien.

3. ¿Qué errores detectó la validación programática que no habías identificado manualmente?

Detectó errores que a simple vista no se notan, como un precio escrito como texto ("28000" en vez de 28000), un booleano entre comillas ("true"), un ID repetido y una fecha fuera del rango. Yo no los habría visto revisando 35 productos a mano; la función los encontró todos y dijo en qué producto y campo estaba cada uno. Además, la revisión con IA encontró fallas de calidad que el código no ve, como unidades escritas distinto y todos los productos marcados como orgánicos.

4. ¿Qué ventajas ofrece separar los datos, la lógica JavaScript y la presentación HTML/CSS?

Si algo falla o quiero cambiarlo, sé en qué archivo buscar. Por ejemplo, pude agregar los filtros y el gráfico en app.js sin dañar el diseño, y la validación está en un solo archivo que uso en varias partes. También es más fácil pedirle a la IA un archivo a la vez y probarlo.

5. ¿Qué limitaciones tiene localStorage frente a una base de datos real?

Los datos solo quedan en mi navegador y en mi computador, así que otra persona no los ve. Tiene un límite de unos 5 MB, no permite hacer consultas ni tener varios usuarios, y se borra si se limpian los datos del sitio. Además, cualquiera puede modificarlo desde el navegador, por eso no sirve para datos importantes.

6. ¿Qué cambios realizarías para convertir el prototipo en una aplicación lista para producción?

Guardaría los productos en una base de datos con un servidor, y pondría un inicio de sesión para que solo el administrador pueda editar. También repetiría la validación en el servidor, agregaría fotos de los productos, un carrito de compras real y pagos, y publicaría la página con HTTPS.

10. Conclusiones

- Dividir el trabajo en fases y archivos pequeños me dio mejores resultados que pedirle toda la aplicación a la IA en un solo prompt.
- La IA me ayudó a ir más rápido con el diseño y los datos, pero tuve que revisar todo y agregar reglas que pudiera comprobar en código.
- Separar datos, lógica y diseño me permitió agregar filtros, tarjetas y el gráfico sin dañar lo que ya funcionaba.
- Aprendí que el control de versiones también hay que revisarlo: una carpeta con su propio .git se sube como submódulo vacío, y con git ls-files comprobé que mis archivos sí estuvieran en el repositorio.